Frequency-Based Replacement for DLRM Embedding Caches: A Trace-Driven Study and a Synthesizable LFU Controller

Adarsh and Pravin Mane
Department of Electrical and Electronics Engineering
BITS Pilani, K K Birla Goa Campus, Goa, India

***Abstract*—Embedding lookups in deep-learning recommendation models (DLRMs) are skewed, which makes caching them attractive, but the replacement policy matters. We compare seven policies by trace simulation on the first 2,000,000 and 10,000,000 rows of the 45,840,617-row Criteo Display Advertising Challenge file, with one cache per table or one cache shared by all 26 tables. With a 256-vector cache per table, both LFU variants beat LRU. With one shared 256-entry cache, LFU that resets counts on eviction beats LRU (47.7% vs. 41.4% hits at 2M rows, 48.2% vs. 41.3% at 10M), while LFU that keeps per-address counts does not (41.0%, 38.5%). ARC and W-TinyLFU are the strongest online policies at small capacities, at the price of extra state. GPU timing of a synthetic 26-table model used CUDA events only, without hardware counters. We also build a synthesizable LFU controller whose default mode belongs to the reset-on-eviction family. On NanGate45 (not TSMC; pre-layout; no power) it reaches 636, 603 and 557 MHz at depths 16, 64 and 256, within a flow noise floor of 4.92–16.23%. Zero-delay gate-level simulation with cell models generated from the liberty file matches the RTL for all 12 netlists; equivalence is proven at depth 16 only.**

I. Introduction

Deep-learning recommendation models combine dense features with many categorical features, each mapped to a row of an embedding table [1]. Characterisation of production recommendation models has shown that these models place different demands on hardware than vision or language networks [2], and a line of systems work exploits the skew of embedding accesses: Bandana keeps embeddings in non-volatile memory with a DRAM cache [3], FAE keeps frequently accessed ("hot") embeddings in GPU memory [4], ScratchPipe builds a GPU-side embedding cache that looks at future accesses [5], and RecNMP moves embedding operations near memory [6]. Each of these needs a rule for which rows to keep. Skew suggests frequency-based replacement (LFU) rather than recency (LRU), but LFU comes in variants that behave differently, and the organisation of the cache (one per table or one for all tables) changes the competition between tables.

This paper reports a trace-driven comparison and a hardware building block, and states the limits of each next to the result:

- A comparison of LRU, two LFU variants, TinyLFU with exact counters, W-TinyLFU, ARC and Belady's offline bound on Criteo traces, for per-table and shared caches, at two trace lengths (Sections III, V).
- The finding that the ranking of the two LFU variants depends on the cache organisation, and that ARC and W-TinyLFU are the strongest online policies at small capacities (Section V).
- GPU timing of embedding lookups without hardware counters, with the launch-bound region identified (Section VI).
- A synthesizable LFU controller, its timing and area on NanGate45 across design iterations with the flow noise floor, and its verification by mutation testing, gate-level simulation and formal equivalence (Sections IV, VII).

Every number in this paper is generated from committed result files (see Artifact Availability).

II. Background

LRU evicts the least recently used entry. **LFU** evicts the entry with the fewest accesses; variants differ in whether an address's count survives its eviction. We call the variant that discards the count on eviction and restarts it at one on re-insertion *LFU, reset on eviction*, and the variant that keeps a per-address count in an unbounded table *LFU, persistent counters*; the latter is an idealised model of history-keeping LFU. **TinyLFU** adds an admission filter that compares the frequency of a new item with that of the eviction candidate; **W-TinyLFU** places a small LRU window in front of the main cache [7] and estimates frequencies with a Count-Min sketch [8]. **ARC** balances a recency list and a frequency list and keeps ghost lists of recently evicted keys to adapt the balance [9]. **Belady's** MIN policy evicts the entry used furthest in the future; it needs the whole trace and serves as an upper bound [10].

III. Trace Simulation Setup

Data. We use the Criteo Display Advertising Challenge training file (Kaggle, 2014) [11], 45,840,617 rows with 26 categorical columns. Each column is one embedding table and each distinct hashed value one vector. We replay the first 2,000,000 and the first 10,000,000 rows, i.e. a prefix of the file; nothing is claimed for the full file. Empty fields produce no access. We do not report Zipf exponents: the committed results hold the fitted exponent per table but not the per-table frequency counts needed to judge the quality of the fit.

Cache organisations. *Per-table*: each table has its own cache of C vectors (26 caches; C from 64 to 100,000). *Shared*: one cache of C entries holds vectors of all tables (key = table and value; C = 256, 1,000, 10,000 and 100,000), and the accesses of

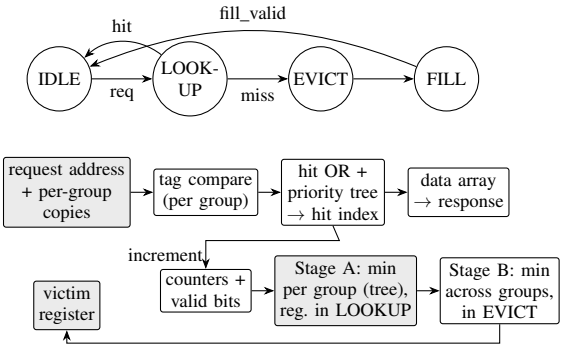


Fig. 1. The LFU controller: FSM (top) and datapath (bottom; shaded blocks are registers). A hit returns to IDLE; a miss selects the victim (Stage A registered in LOOKUP, Stage B in EVICT) and installs the fill in FILL.

TABLE I
HIT RATE (%) WITH 256-VECTOR CACHES, FIRST 2M AND 10M CRITEO ROWS. PER-TABLE: ONE CACHE PER TABLE; SHARED: ONE CACHE FOR ALL TABLES.

	Per-table		Shared	
Policy	2M	10M	2M	10M
LRU	67.3	67.1	41.4	41.3
LFU, reset on eviction	70.1	70.3	47.7	48.2
LFU, persistent counters	71.8	68.3	41.0	38.5
TinyLFU, exact counters	69.7	64.5	–	–
W-TinyLFU (Count-Min)	73.3	73.1	51.0	50.9
ARC	72.6	72.4	50.9	50.9
Belady (offline bound)	78.7	78.4	–	–

–: not run in the shared-cache simulator.

one row are issued in table order 0 to 25. The shared ordering is a modelling choice. Capacities are in vectors, not bytes.

Policies. The seven policies of the previous section. TinyLFU is run with exact counters and a 1% window; W-TinyLFU with a Count-Min sketch and a 1% window. The LFU variants evict the smallest count and break ties in favour of the entry that entered the cache earliest; their counts saturate at 255. The simulators are covered by unit tests and by mutation tests that check the tests fail on deliberately broken policies.

IV. LFU CONTROLLER

Structure. The controller (Fig. 1) stores tags, 128-bit vectors, frequency counters and valid bits in flip-flops (no SRAM macro). Parameters: address width 20, counter width 6 (saturating at 63), 16 groups for victim selection, and a depth of 16, 64 or 256 lines. On a request the FSM goes IDLE→LOOKUP; the tag compare runs per group on registered copies of the request address, and a balanced priority tree gives the lowest matching line. On a hit the counter is incremented and the response is returned. On a miss, Stage A (a reduction tree per group, registered in LOOKUP) and Stage B (a tree across groups, in EVICT) find the line with the smallest count; an invalid line counts as smaller than any valid line. FILL installs the returned vector.

Counter modes. With `COUNTER_MODE` 0 (the default) the count of a newly installed tag is set to one, so an address's history is lost on eviction: the mode corresponds to the reset-on-eviction family of Section III, but it is not the simulated policy. With mode 1 the new tag continues from the count left in the slot. Neither mode keeps a per-address history, so the persistent-counter LFU of the simulations has no RTL counterpart.

Differences from the simulated policy. We compared the tie-break rules in the source code. Among lines with the smallest count, the RTL evicts the one with the lowest slot index (both reduction trees keep the left operand on ties), whereas the simulator evicts the entry that entered the cache earliest. Because a new tag takes the slot of the evicted line, slot index and age differ after the first replacement, so the two can choose different victims. The counters also saturate at different values (63 in the RTL, 255 in the simulator). The RTL was not replayed on the Criteo traces, so the hit rates of Section V are not the RTL's.

Synthesis flow. Yosys 0.33 with ABC [12], [13] maps the RTL to the NanGate45 open cell library (typical corner) [14]; OpenSTA [15] reports setup and hold slack with the library's wire-load model and an ideal clock. NanGate45 is an open-source library for a generic predictive process, not a TSMC process; all results are pre-layout, area is the sum of the cell areas, and no power analysis was run. Each configuration (depth 16, 64, 256; mode 0, 1) is synthesized for a 1 GHz and a 500 MHz target, and $\text{Fmax} = 1/(T - \text{WNS})$ of the worst setup path. The current flow passes the target to ABC through a delay-oriented script whose target is the clock period minus 141.5 ps (clock-to-Q plus setup on the worst path of an earlier depth-16, mode-0 run). The script was chosen on that single depth-16 pilot, and no ablation of its parts was run. To measure the noise of the flow itself we synthesize 8 designs with the same logic (the final RTL, a flat-vector form of it, and three one-line rewrites of each) and report the spread of their Fmax: 1.96–16.62% with the earlier flow and 4.92–16.23% with the current one.

Address copies. The per-group registered copies of the request address were added together with the Stage-B tree and are retained in the final RTL. Their benefit is within the flow noise: at depth 256 the design with copies had 214 vs. 199 MHz without them (mode 0, +7.44%) and 199 vs. 188 MHz (mode 1, +5.84%), less than the 1.96–16.62% spread of the same flow on identical logic, so the copies are not shown to help.

Verification. A self-checking testbench with 7 tests runs on the RTL for every depth and mode, and a lockstep testbench compares each RTL version with earlier versions on random traffic. Mutation testing checks that the tests fail on broken copies of the RTL. For the netlists we run a functional gate-level simulation with zero delay in Icarus Verilog [16]: the cell models are generated by Yosys from the liberty file and checked against its cell functions, and no SDF timing is used. Yosys equivalence checking (`equiv_make`, `equiv_induct`) compares RTL and netlist.

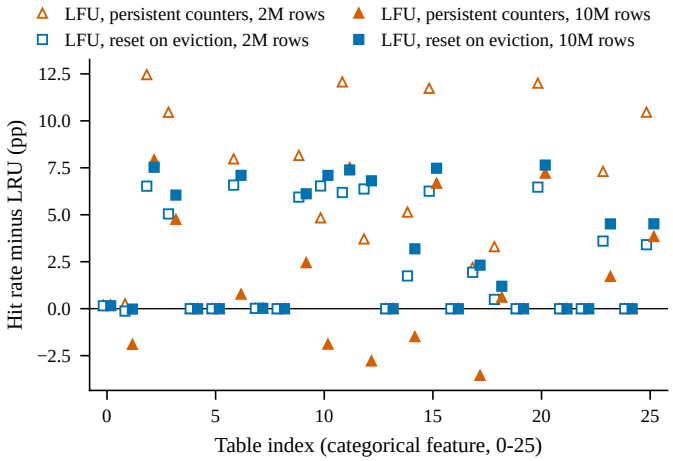


Fig. 2. Per-table caches of 256 vectors: hit rate of each LFU variant minus that of LRU, per table, at 2M (open) and 10M rows (filled). Zero: same hit rate as LRU on that table.

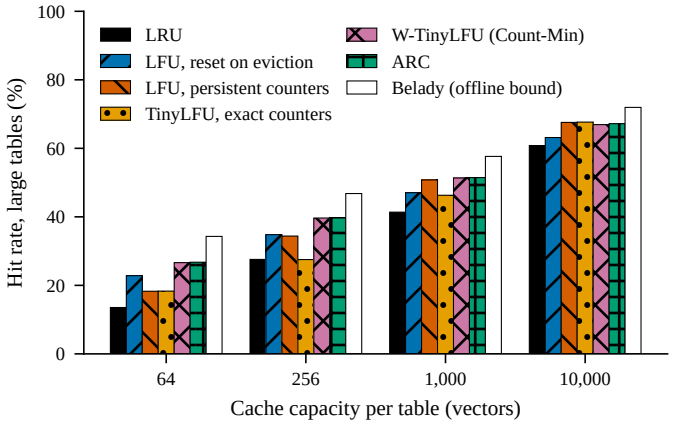


Fig. 3. Per-table caches, the five large tables (2, 3, 11, 15, 20) combined: hit rate by policy and capacity, 10M rows.

V. Results: Replacement Policies

Per-table caches. With 256 vectors per table (Table I), both LFU variants beat LRU at both trace lengths. On all tables combined they beat LRU at every capacity from 64 to 100,000 vectors; the smallest margin is 0.26 (persistent counters) and 0.21 percentage points (reset on eviction) at 2M rows, both at 100,000 vectors, and 0.59 (persistent, at 64 vectors) and 0.40 points (reset, at 50,000 vectors) at 10M rows. The margin of the persistent-counter variant at 256 vectors shrinks with the longer trace (71.8% at 2M, 68.3% at 10M), and per table it falls below LRU on six tables at 10M rows (Fig. 2). TinyLFU with exact counters is below LRU at 10M rows (64.5% vs. 67.1%). W-TinyLFU and ARC are the best online policies at 256 vectors; Belady's bound is 78.7% and 78.4%.

Five tables (2, 3, 11, 15 and 20) have at least 150,000 distinct values at both trace lengths. Fig. 3 shows them combined: at 64 vectors per table, reset-on-eviction LFU, W-TinyLFU and ARC are well above LRU, and Belady's bound stays above every online policy at every capacity shown.

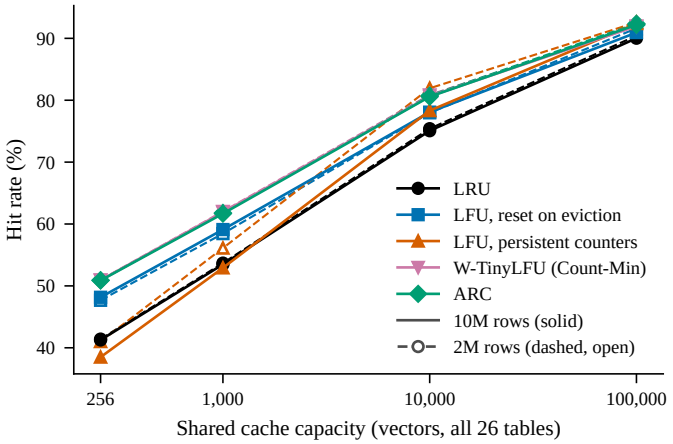


Fig. 4. One cache shared by all 26 tables: total hit rate vs. capacity; 10M rows solid, 2M rows dashed with open markers.

TABLE II
The five large tables inside the shared 256-entry cache: their share of all hits and their own hit rate (%), per policy. They are 20.9% (2M) and 20.9% (10M) of all accesses.

	Share of hits		Hit rate	
Policy	2M	10M	2M	10M
LRU	2.5	2.2	4.9	4.3
LFU, reset on eviction	3.9	4.1	8.9	9.4
LFU, persistent counters	3.6	2.9	7.1	5.3
W-TinyLFU (Count-Min)	4.5	4.4	11.1	10.6
ARC	4.6	4.4	11.3	10.7

Shared cache. With one cache for all tables the picture changes (Fig. 4). At 256 entries, reset-on-eviction LFU beats LRU at both trace lengths (47.7% vs. 41.4% at 2M, 48.2% vs. 41.3% at 10M), and it does so at every capacity tested. The persistent-counter variant is below LRU at 256 entries at both lengths (41.0% and 38.5%) and at 1,000 entries at 10M rows (52.9% vs. 53.5%); it is above LRU at 10,000 and 100,000 entries, and at 100,000 entries it has the highest hit rate of all policies (92.6% and 92.4%). At 256 and 1,000 entries, W-TinyLFU and ARC are the best policies. The persistent-counter variant also loses the most from the shorter to the longer trace.

Large tables in the shared cache. The aggregate shared-cache hit rate hides the large tables (Table II). They make up 20.9% of all accesses at 2M rows and 20.9% at 10M, but contribute only 2.5–4.6% of the hits at 2M and 2.2–4.4% at 10M; their own hit rate is 4.9% (LRU) to 11.3% (ARC) at 2M. Almost all hits of a 256-entry shared cache come from the other 21 tables.

What the comparison shows. The two LFU variants swap places between the two organisations: with per-table caches the persistent-counter variant is ahead at 2M rows, with a shared cache it is behind LRU at small capacity while the reset variant is ahead. In a shared cache, the tables compete for the same entries, and the persistent variant keeps counts for every address it has seen; we did not isolate which of these causes the difference. ARC and W-TinyLFU are the strongest online

policies at small capacities in both organisations, but they need extra state (ghost lists for ARC, a sketch and a window for W-TinyLFU); we did not measure that cost in hardware.

VI. Results: GPU Timing

Setup. A synthetic model with 26 equal EmbeddingBag tables in PyTorch [17] (sum pooling, one lookup per table and sample) followed by an MLP runs fp32 inference on one H100 and one H200. Table rows are 10^6 or 10^7, the dimension 64 or 128, the batch 64 to 8192, with uniform, Zipf and Criteo-hashed indices. Timing uses CUDA events only: the profiler `ncu` was not available on the H100 node and refused to collect counters on the H200 node (`ERR_NVGPUCTRPERM`), so there are no hardware counters, and nothing here shows memory-boundedness or DRAM traffic. On the H100, 40 of 160 configurations were skipped because the 10^7-row, dimension-128 tables did not fit in GPU memory.

Launch-bound region. We call a configuration launch-bound if its CPU wall time per iteration is within 5% of the batch-64 value for the same tables and index pattern. Up to batch 1024 on the H100 and 512 on the H200 every series in Fig. 5 is launch-bound: there the share is a share of kernel-launch time, not of lookup cost. At batch 8192 the embedding share is 0.4224–0.4553 on the H100 and 0.4024–0.4358 on the H200 over all configurations that ran.

Table size and index pattern. Table rows made no visible difference: going from 10^6 to 10^7 rows changed the median embedding time by -1.64% to $+1.38\%$ on the H100 and -1.80% to $+2.07\%$ on the H200, over every configuration that ran at both sizes. The index pattern mattered little, and most at large batches: the spread over the five patterns was 1.94–2.96% (H100) and 2.59–4.31% (H200) at batch 64, and 8.45–10.17% and 7.08–9.68% at batch 8192. The model has 26 tables of equal size; production models have tables of very different sizes, so these numbers are not a statement about any production model.

VII. Results: Controller Hardware

Design iterations. Fig. 6 follows the controller through three RTL changes and one flow change. The baseline reached 203, 195 and 167 MHz at depths 16, 64 and 256 (mode 0). Replacing serial minimum and priority loops by balanced trees raised depth 256 to 451 MHz with the earlier flow; the timing-driven flow then gives 636, 603 and 557 MHz on the same RTL. The noise bands are wide (Section IV), so differences of that size between two designs are not evidence of a design effect.

Final controller. Table III lists the final design. The 500 MHz netlists meet 500 MHz at depths 16 and 64 in both modes. At depth 256 they miss by 0.0357 ns (mode 0) and 0.0015 ns (mode 1); these shortfalls are far inside the noise of 8.53% and 7.02%, so whether depth 256 meets 500 MHz is not demonstrated either way. Compared with the earlier flow on the same RTL, the timing-driven flow uses 1.52% and 1.94% more area at depth 64 and 2.81% and 2.01% more at depth 256, and at depth 64 it has more nets that violate the slew or capacitance limit (2 to 7 in mode 0, 3 to 6 in mode 1; the reset net, which drives every flip-flop before any buffer tree is built, is excluded from these counts). A layout flow would buffer such nets, and timing would change.

TABLE III
Final controller on NanGate45, timing-driven flow, pre-layout; area is the sum of cell areas; no power analysis.

Depth	Mode	Cells	FFs	Area (μm^2)	Fmax	500 MHz	Noise
16	0	11,466	2,957	26,744	636	met	16.23%
16	1	11,057	2,957	26,627	660	met	7.71%
64	0	37,939	10,431	94,386	603	met	6.24%
64	1	38,692	10,431	95,101	611	met	4.92%
256	0	149,620	40,225	369,530	557	-0.0357 ns	8.53%
256	1	150,254	40,225	370,227	542	-0.0015 ns	7.02%

Fmax in MHz from the 1 GHz netlists; FFs = flip-flops. 500 MHz: "met" or the worst negative slack of the netlist synthesized for 500 MHz; at depth 256 the shortfall is far inside the noise, so meeting 500 MHz is not demonstrated. Noise: Fmax spread of 8 same-logic designs.

TABLE IV
Gate-level lockstep stimulus per depth and mode (200,000 random cycles; the 1 GHz and 500 MHz netlists receive the same stimulus) and counter saturation in it, from an RTL rerun of the same stimulus.

Depth	Mode	Hits	Misses	Cycles with a counter at 63	Hit increments suppressed at 63
16	0	57,297	8,139	199,023	55,889
16	1	57,090	8,166	198,775	56,501
64	0	57,092	8,199	196,542	53,072
64	1	53,588	9,369	196,366	51,277
256	0	57,209	8,109	187,876	47,168
256	1	56,927	8,216	187,614	47,838

Depth 16 mode 1 ran 200,002 cycles (the stimulus finishes a fill handshake). Hits and misses are identical in the gate-level runs and the RTL rerun.

Verification. Mutation testing of the final RTL killed 113 mutant runs with the unit tests and 10 more with the lockstep check; 33 runs were of equivalent mutants. On the netlists, the 7 unit tests pass at depths 16 and 64 (Icarus 12) and 16 and 256 (Icarus 11). In the gate-level lockstep, each of the 12 netlists and the RTL see the same 200,000 random cycles; hit flag, response valid, response data, fill request and fill address are compared every cycle, with 0 mismatching cycles and no X on any output. Table IV gives the hits and misses per run. To see whether these runs exercised counter saturation, we reran the same stimulus on the RTL alone and recorded the counters: in every depth and mode the largest count reached is 63, a valid line holds 63 in 187,614 to 199,023 of the cycles, and 47,168 to 56,501 hit increments were suppressed at 63. Saturation is therefore exercised heavily, and most hits in this stimulus do not change a counter. The gate-level simulation has no delays, so it checks the logic of the netlists, not their timing. Yosys proves the four depth-16 netlists equivalent to the RTL (2977 of 2977 equivalence points); the proof treats the asynchronous reset as synchronous, starts the induction from matched registers being equal, and was completed at depth 16 only.

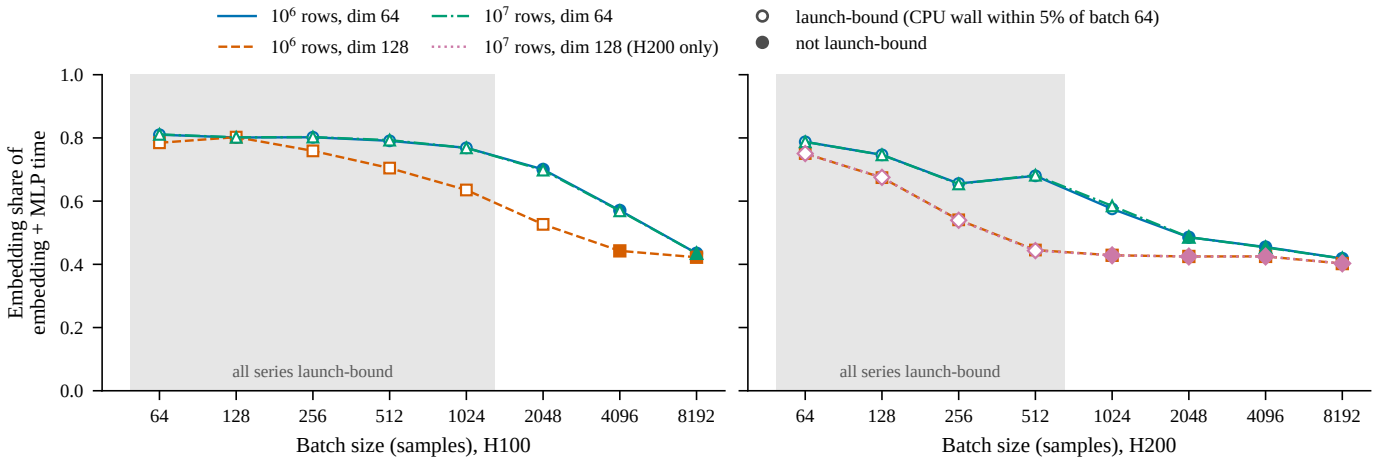


Fig. 5. Median embedding share of embedding + MLP time vs. batch size, Criteo-hashed indices, fp32 inference on one H100 (left) and one H200 (right); CUDA events only. Open markers: launch-bound (CPU wall time per iteration within 5% of that at batch 64). On the H100, 40 of 160 configurations were skipped for memory (10^7 rows, dimension 128).

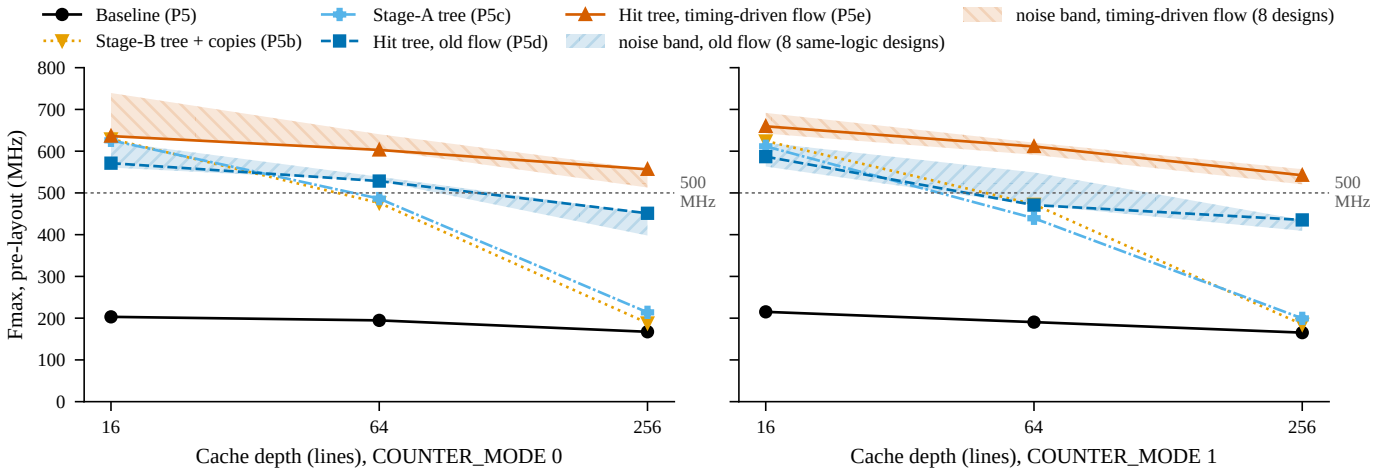


Fig. 6. Pre-layout Fmax on NanGate45 (not TSMC) of the 1 GHz netlists vs. depth for each design iteration: baseline; Stage-B reduction tree with per-group address copies (the copies are retained, but their benefit was within the flow noise floor); Stage-A reduction tree; hit-index priority tree with the earlier flow; the same RTL with the timing-driven flow. Shaded: min–max Fmax of 8 designs with identical logic (flow noise) for the two flows.

VIII. Limitations

- *Data*: only the first 2,000,000 and 10,000,000 rows of the 45,840,617-row file; results changed between the two lengths (most for the persistent-counter LFU), so longer traces may change them again.
- *Organisation*: per-table and shared results are not comparable one to one, and the shared order of accesses within a row is a modelling choice.
- *Simulator vs. RTL*: tie-break and counter width differ (Section IV); the RTL was not replayed on the traces; there is no RTL for persistent per-address counters, ARC or W-TinyLFU.
- *GPU timing*: CUDA events only, no hardware counters; a synthetic model with 26 equal tables and one lookup per table; one GPU of each type; small batches are launch-bound.
- *Hardware*: NanGate45, not TSMC; pre-layout with a wire-load model and ideal clock; area is a sum of cell areas; no power; Fmax differences below the noise floor are not meaningful; the timing-driven flow was tuned on a single depth-16 pilot without ablation, uses up to 2.81% more area, and adds slew/capacitance violations at depth 64.
- *Verification*: gate-level simulation is zero-delay with one random stimulus; equivalence is proven only at depth 16.
- *Earlier work*: an earlier gem5 study of this project (a CPU with DDR4, not a GPU [18]) is not reproduced here; the statistics of most of its runs are not available, and we use none of its numbers.

IX. Conclusion

On Criteo traces, frequency information helps embedding caches, but how it is kept matters, and the answer depends on whether tables share a cache. Per-table caches favour both LFU

variants over LRU; a shared cache favours LFU that resets counts on eviction and penalises the variant that keeps them, at small capacities. ARC and W-TinyLFU are the strongest online policies at small capacities, at a hardware cost we have not measured. The synthesizable controller, whose default mode belongs to the reset-on-eviction family, reaches 542–660 MHz pre-layout on NanGate45 and matches its RTL at gate level. Next steps are replaying the traces through the RTL, longer traces, and a hardware estimate for ARC and W-TinyLFU.

Artifact Availability

RTL, simulators, synthesis and verification scripts, figure scripts and all result files are in https://github.com/idealxdxrsh/Cache-Policy-Phase-Transition-in-DLRM-Workloads. The controller of this paper is git tag `rtl-v2-final`; the first synthesized version is tag `rtl-v2-baseline`. Every number in the paper is in `results/P7/TABLES.md`, generated by `tools/make_tables.py` from committed files only, and `tools/paper_numbers_check.py` checks the text against it. Sources, with the commit that produced them: per-table simulation `results/P3/` (`a84d9a9`); shared cache `results/P3b/` (`c81407a`, `79f9adc`); GPU timing `results/P4/` (`daf1601`); synthesis `results/P5/` to `results/P5e/` (timing-driven flow `37c353c`); gate-level simulation and equivalence `results/P5f_gls/` (`d7a2b54`, `18b6aeb`, `215d493`); counter saturation `results/P7/counter_sat/` (`2af6b00`); figures `paper/v2/figs/`. The Criteo data are not redistributed; their source and checksums are in `results/P2/SOURCE.md`.

References

[1] M. Naumov *et al.*, "Deep learning recommendation model for personalization and recommendation systems," arXiv:1906.00091, 2019.

[2] U. Gupta, C.-J. Wu, X. Wang, M. Naumov, B. Reagen, D. Brooks *et al.*, "The architectural implications of Facebook's DNN-based personalized recommendation," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2020.

[3] A. Eisenman, M. Naumov, D. Gardner, M. Smelyanskiy, S. Pupyrev, K. Hazelwood, A. Cidon, and S. Katti, "Bandana: Using non-volatile memory for storing deep learning models," in *Proc. Machine Learning and Systems (MLSys)*, vol. 1, 2019.

[4] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Accelerating recommendation system training by leveraging popular choices," *Proc. VLDB Endowment*, vol. 15, no. 1, 2022.

[5] Y. Kwon and M. Rhu, "Training personalized recommendation systems from (GPU) scratch: Look forward not backwards," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2022, pp. 860–873.

[6] L. Ke *et al.*, "RecNMP: Accelerating personalized recommendation with near-memory processing," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2020.

[7] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A highly efficient cache admission policy," *ACM Trans. Storage*, vol. 13, no. 4, 2017, article 35.

[8] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[9] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2003, pp. 115–130.

[10] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[11] Criteo Labs, "Display advertising challenge," Kaggle competition, https://www.kaggle.com/competitions/criteo-display-ad-challenge, 2014.

[12] C. Wolf, "Yosys open SYnthesis suite," https://yosyshq.net/yosys/.

[13] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification (CAV)*, ser. LNCS, vol. 6174. Springer, 2010, pp. 24–40.

[14] Nangate, "FreePDK45 and the Nangate 45nm open cell library," Distributed by Si2, https://si2.org/open-cell-and-free-pdk-libraries/.

[15] Parallax Software, "OpenSTA: Parallax static timing analyzer," https://github.com/The-OpenROAD-Project/OpenSTA.

[16] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.

[18] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.